

unknown vector $\mathbf{x}$:

$$\begin{aligned}
 \mathbf{A} \cdot \mathbf{x} &= \mathbf{b} \\
 \mathbf{A} \cdot \mathbf{C}_0 \cdot \mathbf{C}_0^{-1} \cdot \mathbf{x} &= \mathbf{b} \\
 \mathbf{A} \cdot \mathbf{C}_0 \cdot \mathbf{C}_1 \cdot \mathbf{C}_1^{-1} \cdot \mathbf{C}_0^{-1} \cdot \mathbf{x} &= \mathbf{b} \\
 (\mathbf{A} \cdot \mathbf{C}_0 \cdot \mathbf{C}_1 \cdot \mathbf{C}_2 \cdots) \cdots \mathbf{C}_2^{-1} \cdot \mathbf{C}_1^{-1} \cdot \mathbf{C}_0^{-1} \cdot \mathbf{x} &= \mathbf{b} \\
 (\mathbf{I}) \cdots \mathbf{C}_2^{-1} \cdot \mathbf{C}_1^{-1} \cdot \mathbf{C}_0^{-1} \cdot \mathbf{x} &= \mathbf{b}
 \end{aligned} \tag{2.1.7}$$

which (peeling off the $\mathbf{C}^{-1}$'s one at a time) implies a solution

$$\mathbf{x} = \mathbf{C}_0 \cdot \mathbf{C}_1 \cdot \mathbf{C}_2 \cdots \mathbf{b} \tag{2.1.8}$$

Notice the essential difference between equation (2.1.8) and equation (2.1.6). In the latter case, the $\mathbf{C}$'s must be applied to $\mathbf{b}$ in the *reverse order* from that in which they become known. That is, they must all be stored along the way. This requirement greatly reduces the usefulness of column operations, generally restricting them to simple permutations, for example in support of full pivoting.

CITED REFERENCES AND FURTHER READING:

- Wilkinson, J.H. 1965, *The Algebraic Eigenvalue Problem* (New York: Oxford University Press).[1]
 Carnahan, B., Luther, H.A., and Wilkes, J.O. 1969, *Applied Numerical Methods* (New York: Wiley), Example 5.2, p. 282.
 Bevington, P.R., and Robinson, D.K. 2002, *Data Reduction and Error Analysis for the Physical Sciences*, 3rd ed. (New York: McGraw-Hill), p. 247.
 Westlake, J.R. 1968, *A Handbook of Numerical Matrix Inversion and Solution of Linear Equations* (New York: Wiley).
 Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §9.3–1.

2.2 Gaussian Elimination with Backsubstitution

Any discussion of *Gaussian elimination with backsubstitution* is primarily pedagogical. The method stands between full elimination schemes such as Gauss-Jordan, and triangular decomposition schemes such as will be discussed in the next section. Gaussian elimination reduces a matrix not all the way to the identity matrix, but only halfway, to a matrix whose components on the diagonal and above (say) remain nontrivial. Let us now see what advantages accrue.

Suppose that in doing Gauss-Jordan elimination, as described in §2.1, we at each stage subtract away rows only *below* the then-current pivot element. When a_{11} is the pivot element, for example, we divide the row 1 by its value (as before), but now use the pivot row to zero only a_{21} and a_{31} , not a_{01} (see equation 2.1.1). Suppose, also, that we do only partial pivoting, never interchanging columns, so that the order of the unknowns never needs to be modified.

Then, when we have done this for all the pivots, we will be left with a reduced equation that looks like this (in the case of a single right-hand side vector):

$$\begin{bmatrix} a'_{00} & a'_{01} & a'_{02} & a'_{03} \\ 0 & a'_{11} & a'_{12} & a'_{13} \\ 0 & 0 & a'_{22} & a'_{23} \\ 0 & 0 & 0 & a'_{33} \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \end{bmatrix} \quad (2.2.1)$$

Here the primes signify that the a 's and b 's do not have their original numerical values, but have been modified by all the row operations in the elimination to this point. The procedure up to this point is termed *Gaussian elimination*.

2.2.1 Backsubstitution

But how do we solve for the x 's? The last x (x_3 in this example) is already isolated, namely

$$x_3 = b'_3 / a'_{33} \quad (2.2.2)$$

With the last x known we can move to the penultimate x ,

$$x_2 = \frac{1}{a'_{22}} [b'_2 - x_3 a'_{23}] \quad (2.2.3)$$

and then proceed with the x before that one. The typical step is

$$x_i = \frac{1}{a'_{ii}} \left[b'_i - \sum_{j=i+1}^{N-1} a'_{ij} x_j \right] \quad (2.2.4)$$

The procedure defined by equation (2.2.4) is called *backsubstitution*. The combination of Gaussian elimination and backsubstitution yields a solution to the set of equations.

The advantage of Gaussian elimination and backsubstitution over Gauss-Jordan elimination is simply that the former is faster in raw operations count: The innermost loops of Gauss-Jordan elimination, each containing one subtraction and one multiplication, are executed N^3 and N^2m times (where there are N equations and unknowns, and m different right-hand sides). The corresponding loops in Gaussian elimination are executed only $\frac{1}{3}N^3$ times (only half the matrix is reduced, and the increasing numbers of predictable zeros reduce the count to one-third), and $\frac{1}{2}N^2m$ times, respectively. Each backsubstitution of a right-hand side is $\frac{1}{2}N^2$ executions of a similar loop (one multiplication plus one subtraction). For $m \ll N$ (only a few right-hand sides) Gaussian elimination thus has about a factor three advantage over Gauss-Jordan. (We could reduce this advantage to a factor 1.5 by *not* computing the inverse matrix as part of the Gauss-Jordan scheme.)

For computing the inverse matrix (which we can view as the case of $m = N$ right-hand sides, namely the N unit vectors that are the columns of the identity matrix), Gaussian elimination and backsubstitution at first glance require $\frac{1}{3}N^3$ (matrix reduction) + $\frac{1}{2}N^3$ (right-hand side manipulations) + $\frac{1}{2}N^3$ (N backsubstitutions) = $\frac{4}{3}N^3$ loop executions, which is more than the N^3 for Gauss-Jordan. However, the unit vectors are quite special in containing all zeros except for one element. If this